Титульник

Академия топ

Проект по

Основам алгоритмизации

И программированию

Преподаватель: Рычихин Ярослав

Студент:Михайлов Богдан

Номер группы: РПО 23/2

Г. Санкт-Петербург 2025г.

ВВЕДЕНИЕ

Game Library Manager — это десктопное приложение на Python, предназначенное для управления коллекцией компьютерных игр. Программа предоставляет комплексное решение для организации игровой библиотеки, автоматического отслеживания времени игры и хранения полезных ресурсов, связанных с каждой игрой.

В современной игровой индустрии пользователи часто сталкиваются с проблемой управления большими коллекциями игр, разбросанными по разным платформам. Отсутствие централизованного инструмента для отслеживания прогресса, времени игры и связанных ресурсов затрудняет эффективное использование игрового времени. Данный проект решает эти проблемы, предлагая удобный и функциональный инструмент для игроков.

Цель и задачи проекта

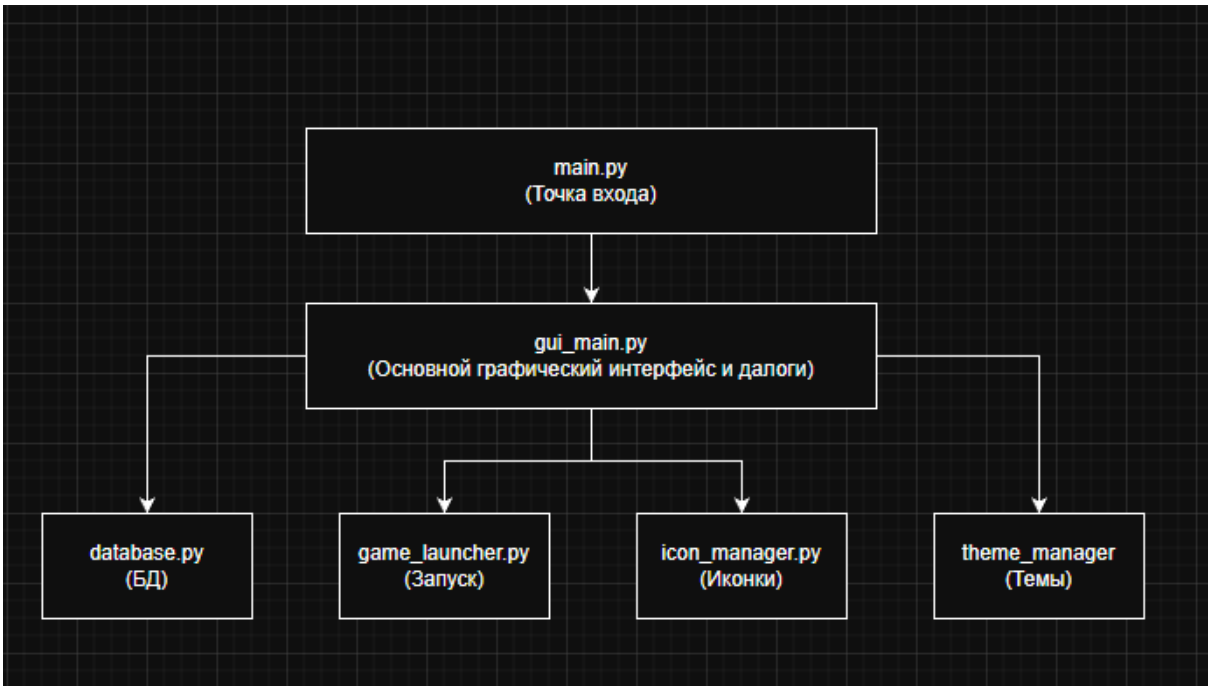
Цель проекта:

Разработка универсального приложения для управления персональной игровой библиотекой, обеспечивающего удобную организацию, автоматическое отслеживание и анализ игрового опыта.

Основные задачи:

- Каталогизация игр: Создание структурированной базы данных для хранения информации об играх с поддержкой всех необходимых атрибутов.
- Автоматическое отслеживание времени: Реализация системы автоматического мониторинга времени игры с точным подсчетом сессий.
- Управление закладками: Создание системы для хранения и организации ссылок на ресурсы, связанные с играми.
- Интуитивный интерфейс: Разработка удобного пользовательского интерфейса с поддержкой темной темы и адаптивной версткой.

Архитектура проекта



Каждый модуль отвечает за определенную функциональность:

- gui_main.py — основной интерфейс пользователя
- database.py — управление данными
- game_launcher.py — запуск и мониторинг игр
- icon_manager.py — работа с иконками
- theme_manager.py — визуальное оформление
- main.py — инициализация приложения

Детальное описание файлов и компонентов

1) requirements.txt

Файл зависимостей содержит все необходимые внешние библиотеки:

pillow>=8.0.0 # Обработка изображений (иконки)

pyperclip>=1.8.0 # Работа с буфером обмена (копирование ключей)

psutil>=5.8.0 # Мониторинг процессов (резерв для будущих функций)

pywin32>=300 # Windows API для извлечения иконок

Назначение: Упрощение установки зависимостей через pip install -r requirements.txt

2) icon_manager.py

Назначение: Извлечение и обработка иконок из исполняемых файлов игр.

Класс IconManager:

Статический класс, не требует создания экземпляра

Все методы реализованы как @staticmethod

Методы:

@staticmethod

def get_icon_image(exe_path, size=(32, 32)):

```
    """
    Основной метод для получения иконки.
    Входные параметры:
    - exe_path: путь к исполняемому файлу
    - size: размер иконки (по умолчанию 32x32)

    Возвращает:
    - ImageTk.PhotoImage объект для использования в Tkinter
    - None в случае ошибки
    """
```

@staticmethod

def create_default_icon(size=(32, 32)):

"""

Создание стандартной иконки синего цвета.

Используется когда не удается извлечь иконку из файла.

"""

Используемые библиотеки:

PIL.Image, PIL.ImageTk: для работы с изображениями

win32ui, win32gui, win32con, win32api: Windows API для извлечения иконок

os: проверка существования файлов

tempfile: временные файлы (в текущей реализации не используется)

3) game_launcher.py

Назначение: Запуск игр и мониторинг их выполнения с отслеживанием времени.

Класс GameLauncher:

Атрибуты:

db: экземпляр DatabaseManager для работы с БД

active_sessions: словарь активных игровых сессий

on_game_end_callbacks: список callback-функций

Методы:

def __init__(self):

"""Инициализация лаунчера с созданием менеджера БД"""

def add_game_end_callback(self, callback):

"""Добавление функции, вызываемой при завершении игры"""

def _notify_game_end(self, game_id, duration):

"""Уведомление всех зарегистрированных callback-функций"""

def launch_game(self, game_id, executable_path):

"""

Запуск игры:

1. Создание записи сессии в БД

2. Запуск процесса через subprocess

3. Добавление в активные сессии

4. Запуск потока мониторинга

"""

def _monitor_process(self, pid):

"""

Мониторинг процесса игры (работает в отдельном потоке):

1. Ожидание завершения процесса

2. Запись времени окончания

3. Уведомление callback-функций

4. Удаление из активных сессий

"""

def get_active_sessions(self):

"""Получение списка активных сессий"""

def force_end_session(self, pid):

"""Принудительное завершение сессии"""

Используемые библиотеки:

subprocess: запуск внешних процессов

threading: многопоточность для мониторинга

time: измерение времени

database.DatabaseManager: работа с базой данных

4) theme_manager.py

Назначение: Управление визуальным оформлением приложения.

Класс ThemeManager:

Константа DARK_THEME:

Словарь с цветовой палитрой темной темы, содержащий 16 цветовых параметров:

- Основные цвета: фон, текст, акцент
- Цвета виджетов: кнопки, поля ввода, таблицы
- Цвета состояний: успех, предупреждение, ошибка

Методы:

```
@classmethod
def apply_dark_theme(cls, root):
    """
    Применение темной темы ко всему приложению:
    1. Установка темы 'clam' для ttk
    2. Настройка стилей для всех виджетов
    3. Применение цветов из DARK_THEME
    """
```

Используемые библиотеки:

tkinter: основной GUI фреймворк

tkinter.ttk: тематические виджеты

5) gui_main.py

Назначение: Основной графический интерфейс пользователя.

Основные классы:

GameLibraryManager (основной класс приложения)

Атрибуты:

- root: главное окно Tkinter
- db: экземпляр DatabaseManager
- launcher: экземпляр GameLauncher
- icon_manager: экземпляр IconManager
- icon_cache: кэш загруженных иконок
- sort_column, sort_reverse: параметры сортировки
- notebook, bookmarks_frame: элементы интерфейса
- last_played_game_id: ID последней запущенной игры

Основные методы:

def __init__(self, root):

"""Инициализация приложения и создание интерфейса"""

def get_game_icon(self, exe_path):

"""Получение иконки игры с кэшированием"""

def create_widgets(self):

"""Создание всех элементов интерфейса"""

def load_games(self):

"""Загрузка игр из БД с отображением иконок"""

def sort_games(self, games):

"""Сортировка списка игр по выбранному критерию"""

def format_play_time(self, seconds):

"""Форматирование времени игры"""

def on_search(self, event):

"""Обработка поиска в реальном времени"""

def update_active_sessions(self):

"""Обновление информации об активных сессиях"""

def on_game_end(self, game_id, duration):

"""Callback-функция при завершении игры"""

Диалоговые окна:

AddGameDialog: Диалог добавления новой игры

EditGameDialog: Диалог редактирования игры (наследует AddGameDialog)

AddBookmarkDialog: Диалог добавления закладки

EditBookmarkDialog: Диалог редактирования закладки (наследует AddBookmarkDialog)

6) main.py

Назначение: Точка входа в приложение.

Функции:

def main():

```
    """
    Основная функция:
    1. Создание главного окна
    2. Центрирование окна на экране
    3. Создание экземпляра GameLibraryManager
    4. Запуск основного цикла событий
    """
```

7) database.py

Назначение: Управление всеми операциями с базой данных SQLite.

Класс DatabaseManager:

Методы:

def __init__(self, db_path='game_library.db'):

"""Инициализация с указанием пути к БД"""

def init_database(self):

"""

Создание структуры базы данных:

- Таблица games (игры)

- Таблица play_sessions (сессии)

- Таблица bookmarks (закладки)

- Индексы для оптимизации

"""

Методы работы с играми:

def add_game(self, name, platform, executable_path, status, activation_key, notes):

"""Добавление новой игры"""

def get_all_games(self, sort_by='name', reverse=False):

"""Получение всех игр с сортировкой"""

def update_game(self, game_id, name, platform, executable_path, status, activation_key, notes):

"""Обновление информации об игре"""

def delete_game(self, game_id):

"""Удаление игры и связанных данных"""

Методы работы с сессиями:

def start_play_session(self, game_id):

"""Начало игровой сессии"""

def end_play_session(self, session_id):

"""Завершение сессии с подсчетом времени"""

Методы работы с закладками:

def add_bookmark(self, game_id, title, url, description, category):

"""Добавление закладки"""

def get_bookmarks_by_game(self, game_id):

"""Получение закладок по игре"""

Статистические методы:

def get_play_stats(self, game_id):

"""Статистика по игре"""

def get_total_play_time_stats(self):

"""Общая статистика библиотеки"""

Ключевые алгоритмы

1) Алгоритм отслеживания времени игры

Расположение: game_launcher.py, методы launch_game() и _monitor_process()

Поток выполнения:

```
# Основной поток
1. session_id = db.start_play_session(game_id)  # Запись в БД
2. process = subprocess.Popen(executable_path)  # Запуск игры
3. threading.Thread(target=_monitor_process, args=(process.pid,)).start()

# Поток мониторинга
1. process.wait()  # Ожидание завершения
2. duration = db.end_play_session(session_id)  # Запись времени
3. _notify_game_end(game_id, duration)  # Уведомление
```

Особенности:

- Многопоточность для неблокирующего мониторинга
- Точное измерение времени через системные часы
- Автоматическая запись в базу данных

2) Алгоритм сортировки и поиска

Алгоритм сортировки (gui_main.py, метод sort_games()):

```
def sort_games(self, games):
    if self.sort_column == 'name':
        return sorted(games, key=lambda x: x[1].lower(),
reverse=self.sort_reverse)
    elif self.sort_column == 'status':
        status_order = {'Не пройдена': 0, 'В процессе': 1, 'Пройдена': 2,
'Отложена': 3}
```

```
        return sorted(games, key=lambda x: status_order.get(x[4], 4),
reverse=self.sort_reverse)
    # ... другие критерии
```

Особенности:

- Пользовательский порядок сортировки статусов
- Сортировка без учета регистра для названий
- Поддержка обратной сортировки через reverse=True

Алгоритм поиска (gui_main.py, метод on_search()):

```
def on_search(self, event):
    query = self.search_var.get().lower()
    for item in self.tree.get_children():
        game_name = self.tree.item(item)['values'][2].lower()
        if query in game_name:
            self.tree.selection_set(item)  # Выделение
            self.tree.see(item)            # Прокрутка к элементу
```

3) Алгоритм извлечения иконок из EXE файлов

Расположение: icon_manager.py, метод get_icon_image()

Последовательность действий:

1. Проверка существования файла: os.path.exists(exe_path)
2. Извлечение иконок через Windows API: win32gui.ExtractIconEx(exe_path, 0)
3. Обработка первой крупной иконки: использование icon_handle = large_icons[0]
4. Создание совместимого контекста устройства:

```
hdc = win32ui.CreateDCFromHandle(win32gui.GetDC(0))
hbmp = win32ui.CreateBitmap()
hbmp.CreateCompatibleBitmap(hdc, size[0], size[1])
```

5. Рисование иконки на Bitmap: win32gui.DrawIconEx()

6. Преобразование в PIL Image: Image.frombuffer()
7. Конвертация в формат Tkinter: ImageTk.PhotoImage(img)
8. Очистка ресурсов: win32gui.DestroyIcon()

Особенности:

- Кэширование загруженных иконок в icon_cache
- Fallback на стандартную синюю иконку при ошибках
- Обработка исключений для всех этапов

4) Алгоритм работы с базой данных

Транзакция завершения сессии (database.py, метод end_play_session()):

```
def end_play_session(self, session_id):
    # 1. Получение данных о сессии
    cursor.execute('SELECT game_id, start_time FROM play_sessions WHERE id=?', (session_id,))

    # 2. Расчет продолжительности
    duration = int((end_time - start_dt).total_seconds())

    # 3. Атомарное обновление двух таблиц
    cursor.execute('UPDATE play_sessions SET end_time=?, duration=? WHERE id=?',
                   (end_time, duration, session_id))
    cursor.execute('UPDATE games SET total_play_time = total_play_time + ? WHERE id=?',
                   (duration, game_id))

    # 4. Фиксация изменений
    conn.commit()
```

Особенности:

- Атомарность операций для сохранения целостности данных

- Использование индексов для оптимизации запросов
- Параметризованные запросы для защиты от SQL-инъекций

6) Алгоритм форматирования времени

Расположение: gui_main.py, метод format_play_time()

```
def format_play_time(self, seconds):
    if not seconds:
        return "0м"

    hours = seconds // 3600
    minutes = (seconds % 3600) // 60

    if hours > 0:
        return f"{hours}ч {minutes}м"  # Пример: "5ч 30м"
    else:
        return f"{minutes}м"           # Пример: "45м"
```

Особенности:

Адаптивный формат в зависимости от величины

Обработка нулевых значений

Читаемый вывод для пользователя

Примеры работы

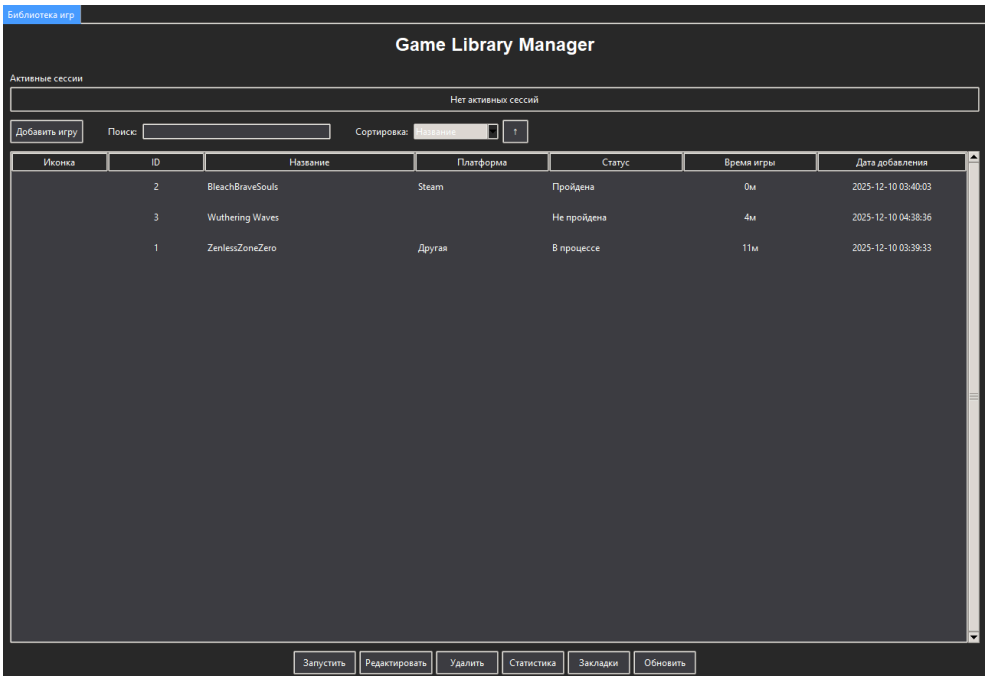


Рис.1. Главное меню приложения

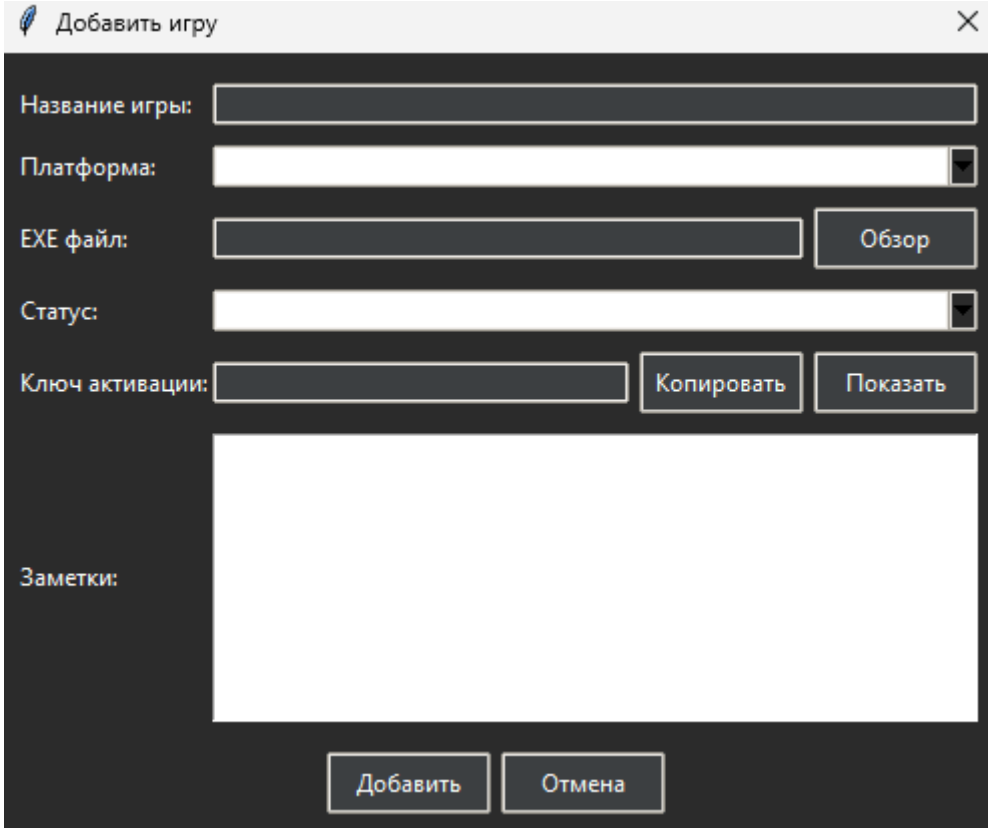


Рис. 2. Меню добавления игры

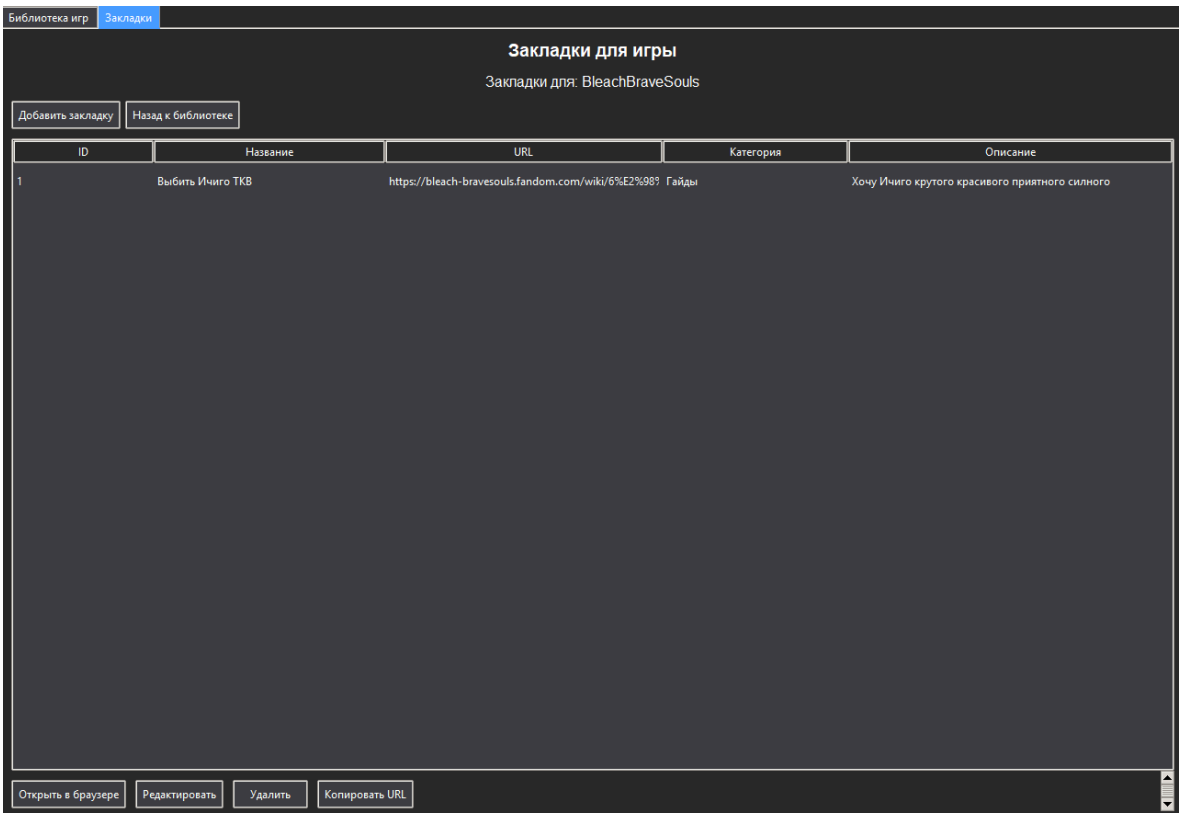


Рис. 3. Закладка для игры

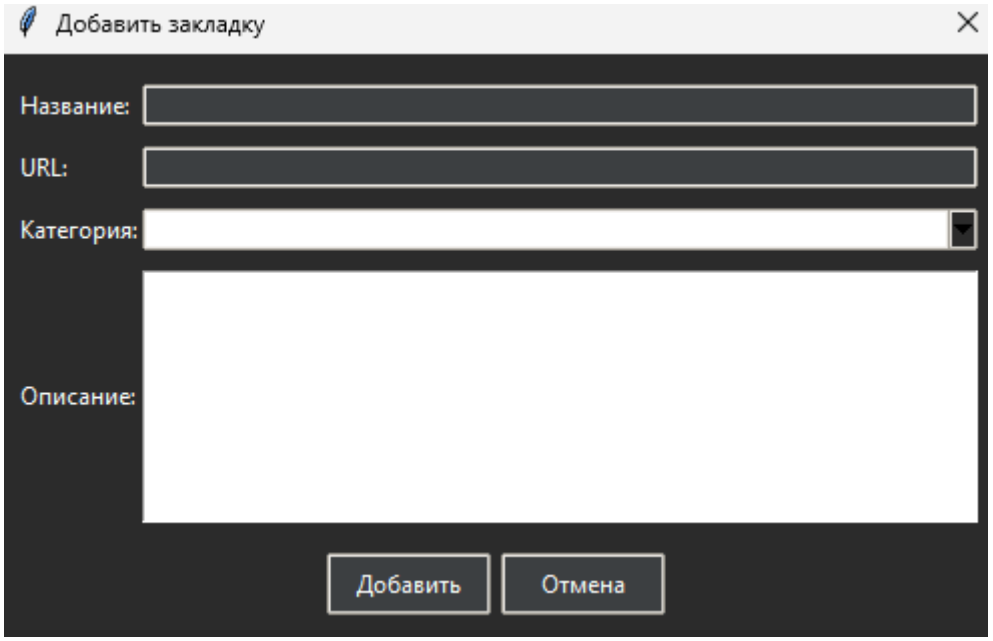


Рис. 4. Меню добавления закладки для игры

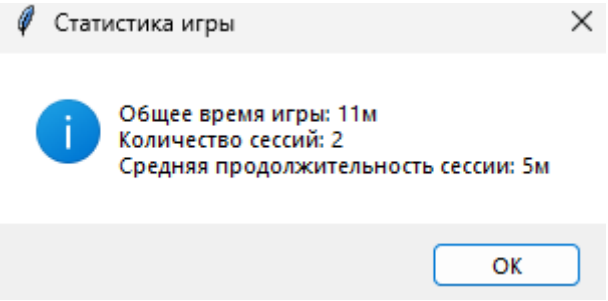


Рис. 5. Меню статистики игры

Заключение

Достигнутые результаты:

1. Полнофункциональное приложение: Реализованы все запланированные функции управления игровой библиотекой.
2. Автоматическое отслеживание: Система автоматически отслеживает время игры без вмешательства пользователя.
3. Удобный интерфейс: Разработан интуитивно понятный интерфейс с поддержкой темной темы.
4. Надежное хранение данных: Использование SQLite обеспечивает стабильное хранение и быстрый доступ к данным.

Технические преимущества:

1. Модульная архитектура: Четкое разделение ответственности между компонентами.
2. Многопоточность: Фоновый мониторинг процессов не блокирует интерфейс.
3. Кэширование: Оптимизация производительности через кэширование иконок.
4. Обработка ошибок: Комплексная обработка исключительных ситуаций.
5. Расширяемость: Архитектура позволяет легко добавлять новые функции.

СПИСОК ИСТОЧНИКОВ

https://docs-python.ru/standart-library/modul-threading-python/klass-thread-modulja-threading/

https://habr.com/ru/sandbox/182102/

https://metanit.com/python/tkinter/

https://habr.com/ru/articles/754400/

https://sky.pro/wiki/python/kopirovanie-stroki-v-bufer-obmena-na-python-shag-za-shagom/

ПРИМЕЧАНИЕ

https://github.com/Drug757/GameLibraryManager